

Loan Default Prediction: an End-to-End Production ML Pipeline

A fully automated, Airflow-orchestrated pipeline that trains and selects a credit-default model, scores every monthly cohort of loan applications, and monitors model performance and stability as labels mature — all running in Docker, end to end, on top of a medallion datamart.

0.78

average AUC across 6 production cohorts
(OOT AUC at selection: 0.80)

3,015

loan applications scored Jan–Jun 2024,
stored as a gold table

< 0.06

PSI on score distribution every month —
no drift vs training baseline

24

backfilled monthly DAG runs: warm-up →
train → infer → monitor

Kevin Aldrin Tan • June 2026 • audience: engineering, data science & business stakeholders

Business context & what the pipeline delivers

The business problem

The bank issues cash loans. Credit losses come from borrowers who stop paying. We need a **default risk score at the point of application**, so risky applications can be declined, repriced, or routed to manual review — before money goes out the door.

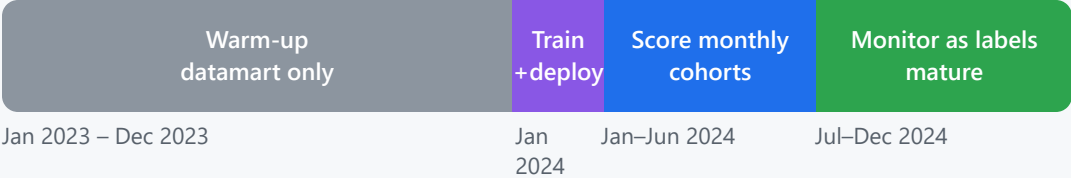
What "default" means here

A loan is labelled **bad** if it is ever **30+ days past due at month-on-book 6** (`30dpd_6mob`). This is the label definition already used by the upstream datamart's label store; observed bad rate $\approx$ **29%**.

Key consequence: labels arrive late

We only learn whether a January application was good or bad **6 months later**. The entire pipeline — training windows, monitoring cadence — is designed around this delay.

One model lifecycle, replayed month by month



- **Warm-up (2023):** applications accumulate and their labels mature — nothing to deploy yet.
- **Train (Jan 2024):** first date with 7 months of fully matured labels; 3 candidate models trained, best one registered in the model bank.
- **Inference (Jan–Jun 2024):** each month's ~500 new applications scored by the deployed model.
- **Monitoring (Jul–Dec 2024):** each month, the cohort scored 6 months earlier gets its actual outcomes — performance & drift are measured and stored.

Delivered: a risk score for every application, plus a monthly, auditable record of how well those scores are working.

Datamart: adopt a proven upstream pipeline, don't rebuild it

Source	Contents	Grain
features_attributes	age, occupation (PII dropped in gold)	1 row / applicant
features_financials	income, debt, credit history, payment behaviour	1 row / applicant
feature_clickstream	20 behavioural web features	monthly / customer
lms_loan_daily	loan book: installments, overdue amounts	monthly / loan

The choice

Three classmate datamarts (simulating other tech families) were available alongside my own Assignment 1 pipeline. I adopted the **LAM NGUYEN THANH THAO datamart end-to-end** — bronze → silver → gold, code unchanged — and built the ML pipeline as its downstream consumer.

Why this one

- **Point-in-time correctness built in:** its gold layer joins each feature snapshot to labels exactly 6 months ahead (multi-snapshot join) — the hardest part of leakage-safe credit data, already done and documented.
- **ML-ready outputs:** a single feature store (72 features after encoding: financial ratios, capped outliers, one-hot categoricals, clickstream) and a clean label store ([30dpd_6mob](#)).
- **Realistic org design:** in a bank, the feature platform is owned by another team. Consuming it as-is, contract-style, mirrors production reality and keeps one source of truth.

Known trade-offs (documented, accepted)

- Outlier caps & one-hot encoders in the gold layer are fitted on the full history — a mild contamination we tolerate as datamart consumers; **all ML preprocessing (imputation, scaling) is refitted on the training window only.**
- Feature store covers applicants who took a loan (labelled population) — consistent for train, inference and monitoring.

Architecture: one monthly Airflow DAG, fully Dockerised



DAG task (runs monthly, backfillable)	What it does	Active when
<code>build_datamart</code>	Full bronze→silver→gold rebuild (idempotent; skipped once built)	first run / missing
<code>gate_training → train_and_select_model</code>	Train 3 candidates, select by OOT AUC, register in model bank	Jan 2024
<code>gate_inference → run_inference</code>	Score the month's cohort with the active model	Jan–Jun 2024
<code>gate_monitoring → run_model_monitoring</code>	Evaluate cohort from 6 months ago; refresh charts	Jul–Dec 2024

Short-circuit gates make every monthly run cheap and self-describing: a single `airflow dags backfill` (or unpausing with catch-up) replays the entire model lifecycle deterministically.

Packaging & reproducibility

- **docker-compose up** → Postgres + Airflow webserver/scheduler at `localhost:8080` (admin/admin).
- Custom image: Airflow 2.9 + Java (PySpark for the datamart) + pinned `requirements.txt`.
- `data/`, `datamart/`, `model_bank/`, `utils/` volume-mounted — outputs persist on the host and every artefact is inspectable.
- Scales by swapping LocalExecutor for Celery/K8s; no code change.

No leakage: every decision uses only what was knowable that day

The trap we designed against

A label for an application made in month m only exists at $m+6$. Training "as of Jan 2024" on Dec 2023 applications would silently use loan outcomes from Jun 2024 — **temporal leakage** that inflates offline metrics and collapses in production.

Guardrails implemented

- **Maturity rule:** the Jan 2024 training run may only use application cohorts $\leq$ Jul 2023 (labels fully observed by Jan 2024).
- **Out-of-time selection:** models are compared on the two most recent matured cohorts (Jun–Jul 2023), not a random split — mimicking how the model will actually be used: trained on the past, applied to the future.
- **No test contamination:** imputer & scaler are fitted on the training split only, inside an sklearn Pipeline, then frozen into the artefact.
- **Target isolation:** labels live in a separate gold store; features never include outcome columns.

Data windows (application cohorts)

Window	Cohorts	Used for
TRAIN	Jan–May 2023 (2,568 rows)	fit candidates (80/20 internal split)
OOT	Jun–Jul 2023 (988 rows)	model selection — untouched by fitting
SCORE	Jan–Jun 2024 (3,015 rows)	production inference
MONITOR	same cohorts, +6 months	actuals vs predictions

Aug–Dec 2023 cohorts are deliberately unused for training: their labels had not matured by the Jan 2024 training date. Leaving them out is the cost of honesty — and the OOT AUC (0.80) matched production AUC (0.75–0.81), evidence the discipline paid off.

Result: offline selection metrics predicted live performance within ± 0.05 AUC — the pipeline's numbers can be trusted for governance decisions.

Train three candidates, let out-of-time data pick the winner

Candidate	Train AUC	Test AUC	OOT AUC ▼	Verdict
Logistic Regression (scaled, median-imputed)	0.806	0.779	0.797	SELECTED
HistGradientBoosting	1.000	0.791	0.785	overfits; small gain not robust OOT
Random Forest (300 trees)	0.987	0.778	0.784	overfits; weakest OOT

Selection rule: **max OOT AUC**. The boosted trees win on the random test split but lose out-of-time — exactly the gap OOT validation exists to expose. With ~2.5k training rows and 72 features, the regularised linear model generalises best across months.

What gets registered in the model bank

- `model.pkl` — frozen sklearn pipeline (imputer → scaler → model), feature list, PSI baseline (training score deciles).
- `metadata.json` — train/OOT windows, row counts, **all** candidate metrics, selection rule, label definition, deployment date.

Versioning & rollback

Artefacts are immutable, named `credit_model_<deploy-date>`. Inference resolves "the active model" as the latest version deployed on/before the scoring date — so re-running history always uses the model that was live at the time, and rollback is just re-pointing to a previous version.

Audit-ready:

every prediction row carries its `model_version`; every version carries the evidence for why it was chosen.

Why we're comfortable shipping a linear model

- Stable: train→OOT AUC drop of 0.01 vs 0.20+ for the tree models.
- Interpretable coefficients — easier conversations with credit risk & compliance.
- Cheap to retrain monthly; deterministic (fixed seed) and fully reproducible.

Predictions & monitoring live as gold tables in the datamart

gold/model_predictions — one row per scored application

Column	Example
Customer_ID	CUS_0x66b3
snapshot_date (partition)	2024-03-01
model_version	credit_model_2024-01-01
default_probability	0.34
default_prediction (p ≥ 0.5)	0

Hive-style partitions per month → cheap incremental writes, trivially queryable by Spark / pandas / BI tools. Idempotent: re-running a month overwrites that partition only.

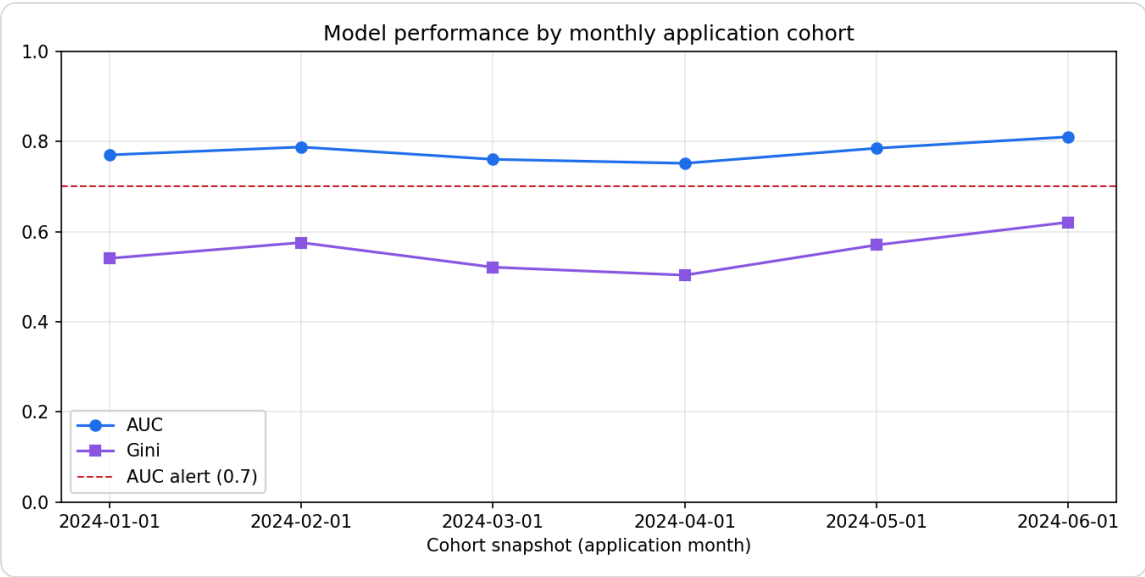
gold/model_monitoring — one row per cohort, written when its labels mature

Group	Columns
Identity	cohort_snapshot_date, monitored_at, model_version, n_scored, n_labeled
Performance	auc, gini, accuracy, precision, recall, f1
Calibration	predicted_default_rate, actual_default_rate
Stability	psi (scores vs training baseline)
Governance	status ∈ OK / WATCH / ALERT

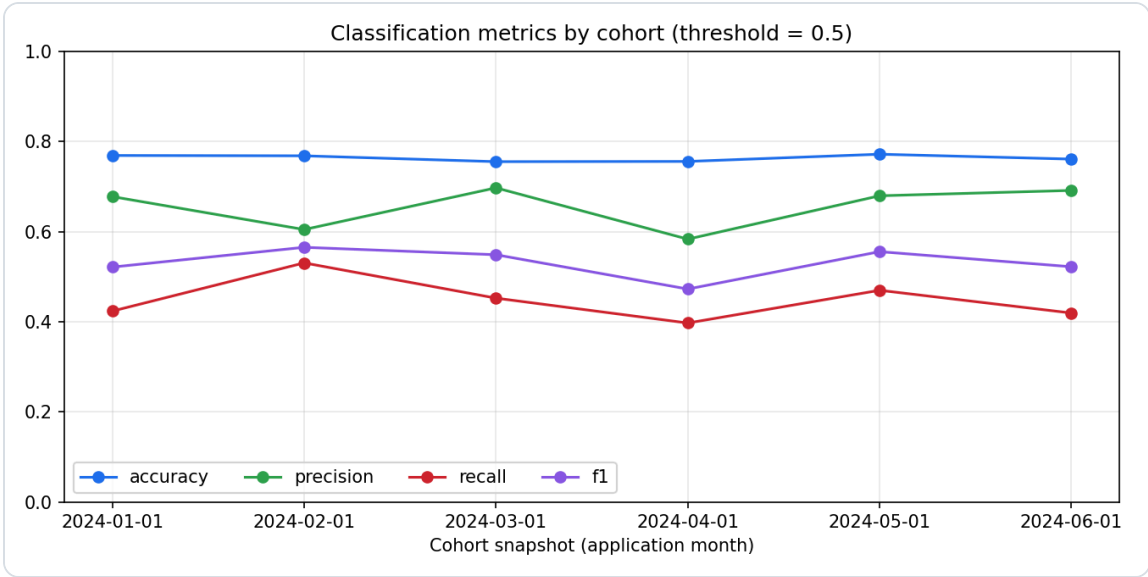
The monitoring task also regenerates the four trend charts (gold/monitoring_viz/) on every run, so dashboards and this deck always show the latest state.

Single source of truth: business reads scores from one table, risk reads health from the other — no side-channel spreadsheets.

Discrimination holds at AUC 0.75–0.81 across all six cohorts



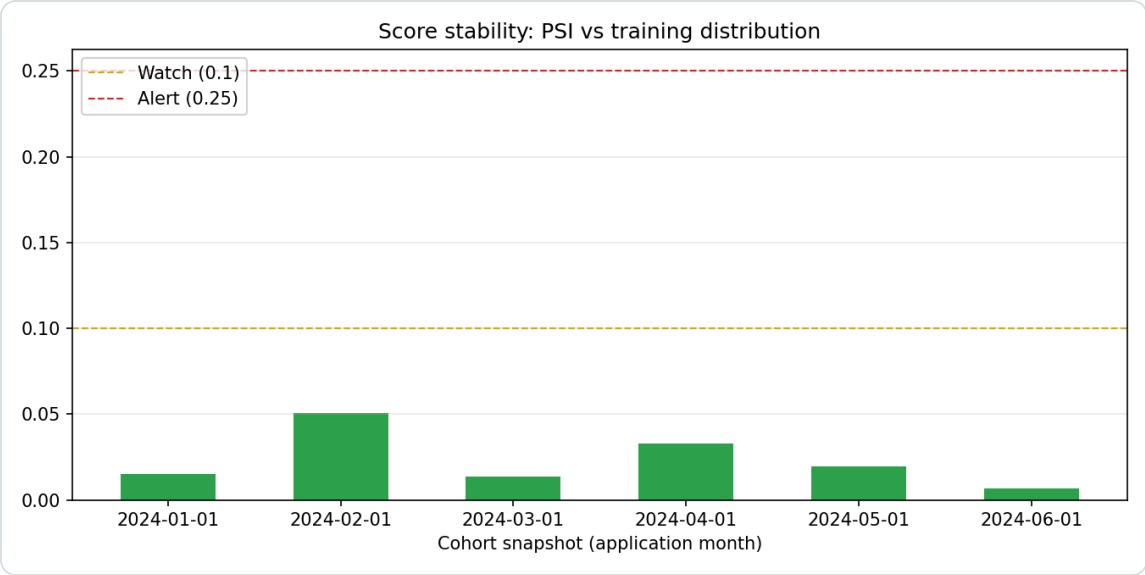
AUC / Gini by application cohort. Production AUC stays in 0.75–0.81, bracketing the 0.80 OOT estimate from selection, and never approaches the 0.70 alert line. Mild month-to-month wobble is consistent with ~500-application cohort sizes.



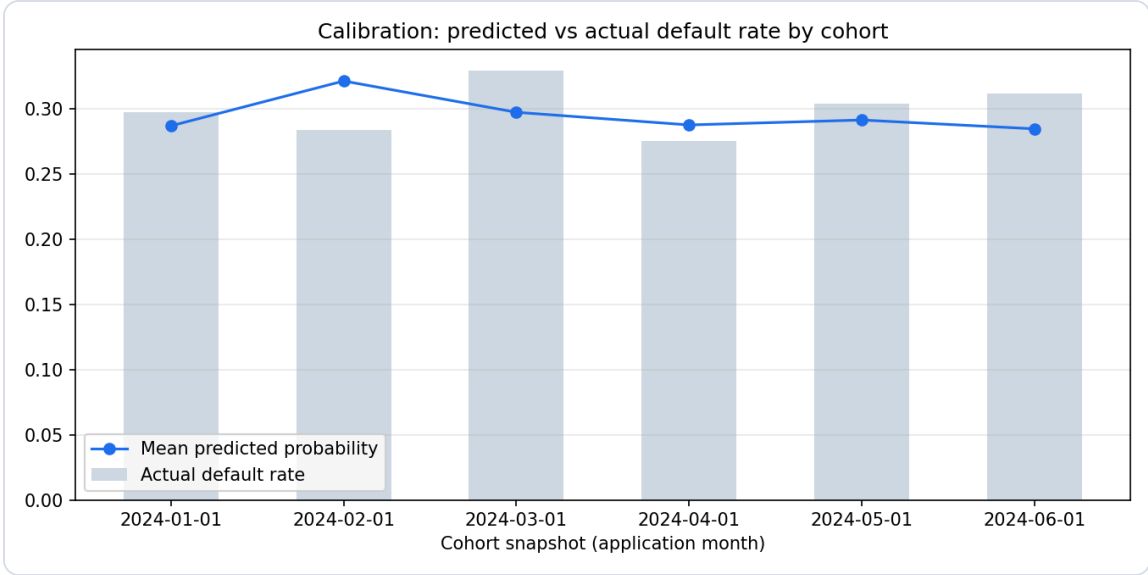
Decision metrics at the 0.5 threshold. Accuracy $\approx$ 0.76–0.77 and precision $\approx$ 0.58–0.70; recall sits at 0.40–0.53 — the conservative threshold favours precision (fewer good customers wrongly declined). Threshold tuning against approval-rate and loss targets is the natural next business conversation.

Read: six months after deployment, the model still ranks risk essentially as well as it did at selection time — no retrain trigger from performance.

Scores are stable (PSI ≤ 0.05) and track actual default rates



Population Stability Index of each cohort's score distribution vs the training baseline (decile bins). All six months sit at 0.01–0.05, far below the 0.10 watch and 0.25 alert thresholds — the applicant population the model sees is the population it was trained on.



Calibration. Mean predicted probability (line) tracks the realised default rate (bars) within ~4pp every month — scores can be read as approximate probabilities, which matters for pricing and expected-loss provisioning, not just rank-ordering.

Read: no drift, no miscalibration → monitoring status OK for all cohorts; the model stays in production under the SOP on the next slide.

Model governance: clear thresholds, a refresh SOP, and a path to scale

Monitoring policy (automated monthly)

Signal	OK	Watch	Alert
AUC (matured cohort)	≥ 0.75	0.70–0.75	< 0.70
PSI (scores vs training)	< 0.10	0.10–0.25	> 0.25
Calibration gap	$< 5\text{pp}$	5–10pp	$> 10\text{pp}$

Status is written into the gold monitoring table by the pipeline itself — the audit trail is the data, not a wiki page.

Refresh SOP

- **Scheduled:** retrain every 6 months on the expanded matured window; challenger must beat champion on OOT AUC to be promoted.
- **Triggered:** any ALERT, or two consecutive WATCH months → immediate retrain + root-cause review with credit risk.
- **Promotion = new model-bank version;** old versions retained for rollback and for replaying history.

Deployment options

- **Today — batch (chosen):** monthly Airflow scoring into the gold table; matches the monthly application snapshot cadence and is the simplest thing that works.
- **Next — on-demand API:** wrap the same model-bank artefact in a FastAPI service for real-time decisioning at application submit; the artefact + version contract already supports it.
- **Scale:** swap LocalExecutor → Celery/Kubernetes executor; datamart already runs on Spark.

Roadmap

- Threshold tuning with the business (approval rate vs expected loss).
- Feature-level drift (CSI) on top of score-level PSI.
- Champion–challenger shadow scoring before each promotion.

Bottom line: the pipeline doesn't just produce a model — it produces the evidence, controls and escape hatches needed to run one responsibly.